Contents

How requirements differ from specification?

The key distinction is that **traditional requirements describe what the system should do**, while a modern **specification describes the system precisely enough that an AI agent can actually construct, test, and verify it**.

This distinction becomes especially important in the 2026 development model you were outlining:

> **Problem → Specification → Context → Agent(s) → Generated implementation → Automated verification**

1. Traditional requirements

A traditional requirement is primarily a **statement of desired behavior or business need**.

For example:

> "Users should be able to upload a PDF and ask questions about it."

Or, more formally:

> "The system shall allow authenticated users to upload PDF documents up to 50 MB."

Requirements typically answer:

- **What** capability is needed?
- **Why** is it needed?
- **Who** needs it?

- What are the major constraints?

They intentionally leave substantial implementation freedom to engineers.

A requirements document might contain:

```
REQ-17:
The system shall allow users to search uploaded documents.

REQ-18:
Search results shall be relevant to the user's query.

REQ-19:
The system shall respond within 3 seconds under normal load.
```

These are useful, but an AI coding agent still has to infer a great deal.

---

2. A specification is more operational

A specification takes the requirement and turns it into something closer to an **executable engineering contract**.

For example:

Requirement

> Users should be able to search their uploaded documents.

Specification

```
Document Search

Input:
  query: string, 1-500 characters
  user_id: authenticated UUID
  filters:
    document_ids: optional list[UUID]
    date_range: optional

Processing:
  1. Normalize query.
  2. Generate embedding using model X.
  3. Perform hybrid BM25 + vector retrieval.
  4. Retrieve top 50 candidates.
  5. Rerank candidates using model Y.
  6. Return top 8 passages.
```

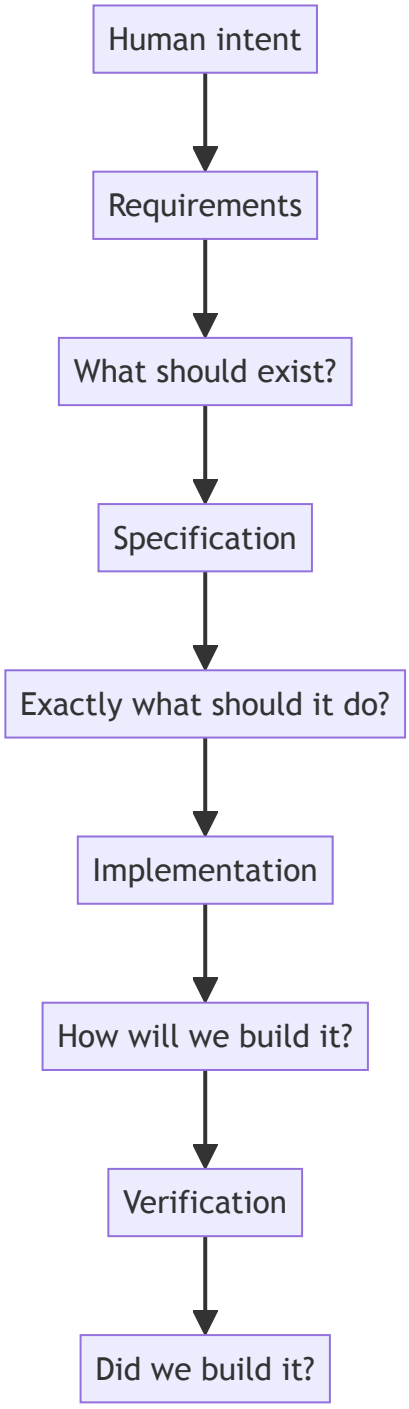
```
Output:
  {
    results: [
      {
        document_id: UUID,
        chunk_id: UUID,
        text: string,
        score: float,
        citation: Citation
      } ]
  }

Constraints:
  - p95 latency < 800 ms
  - no results from documents the user cannot access
  - deterministic filtering
  - scores must be normalized to [0,1]

Failure behavior:
  - malformed query → 400
  - unauthorized document → excluded
  - retrieval service unavailable → 503
  - zero matches → empty result set

Tests:
  - authorization isolation
  - exact-match retrieval
  - semantic retrieval
  - empty query
  - 50 MB document
  - retrieval timeout
  - latency p95
```

Now an AI agent has substantially less to guess.

---

3. The deepest difference: requirements express intent; specifications reduce ambiguity

You can think of the relationship this way:


Human intent
Requirements
What should exist?
Specification
Exactly what should it do?
Implementation
How will we build it?
Verification
Did we build it?

Traditional engineering often treats requirements as the beginning of a **human interpretation process**.

AI-native engineering needs to make that interpretation much more explicit.

---

4. Specifications contain things requirements traditionally leave implicit

A useful specification generally makes several dimensions explicit:

Dimension	Requirement	Specification
Intent	Yes	Yes
Functional behavior	Yes	Yes
Interfaces	Sometimes	Explicit
Data structures	Sometimes	Explicit
Preconditions	Rarely	Explicit
Postconditions	Sometimes	Explicit
Error behavior	Often vague	Explicit
Edge cases	Limited	Explicit
Constraints	High-level	Precise
Security properties	General	Testable
Performance	Targets	Measurable conditions
State transitions	Rarely	Explicit
Dependencies	General	Explicit
Acceptance criteria	Yes	Executable/testable
Tests	Separate artifact	Often derived directly
Implementation	Usually unspecified	Still can remain abstract

The important point is that **a specification does not necessarily prescribe implementation**.

It can say:

> "The operation must be idempotent."

without saying:

> "Use PostgreSQL advisory locks."

That distinction remains valuable.

---

5. Specification is not simply "more detailed requirements"

This is an important nuance.

You don't want to turn every requirement document into a 500-page implementation document.

The difference is **precision and formalizability**, not merely length.

Consider:

> "The application should be fast."

Adding more words doesn't make this a better specification.

Instead:

> "For requests containing <=10,000 records, p95 end-to-end latency must be <500 ms at 100 requests/sec."

Now you have a **verifiable property**.

Likewise:

> "The system should handle errors gracefully."

becomes:

```
If the downstream payment service returns 429:
  retry with exponential backoff
  maximum attempts = 3
  maximum elapsed retry time = 5 seconds
  return PAYMENT_PROVIDER_UNAVAILABLE if all attempts fail
  do not create a duplicate order
```

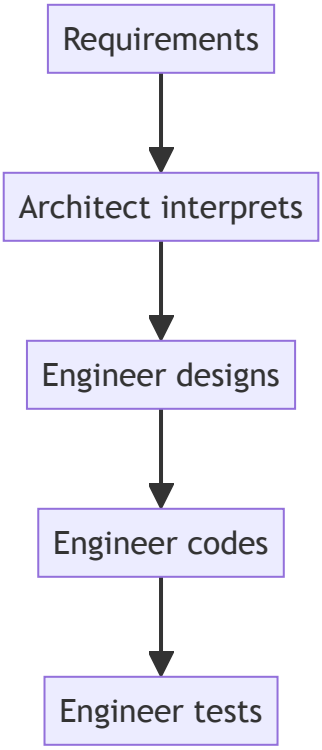
The second version is useful to both a human engineer **and an AI agent**.

---

6. The crucial new dimension: specifications become agent context

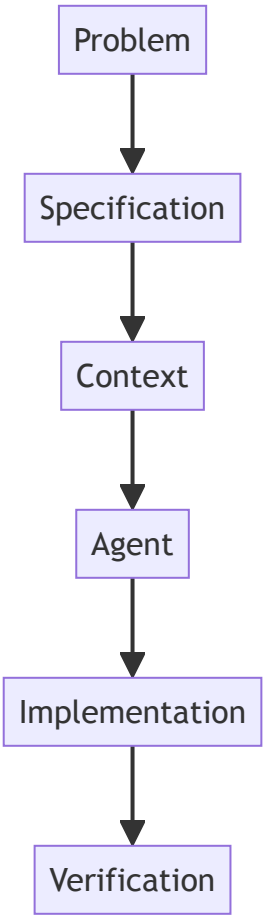
This is where the distinction becomes particularly important for your **AI Engineer's future** chapter.

In traditional development:



There are several layers of implicit human reasoning.

In AI-assisted development:



The specification becomes a major part of the **context supplied to the agent**.

The better the specification, the smaller the agent's inference space.

You can almost think of it as:

$$\text{Implementation quality} \approx f(\text{model capability}, \text{context quality}, \text{specification precision}, \text{verification})$$

A vague requirement forces the agent to make architectural and behavioral assumptions.

A precise specification constrains those assumptions.

---

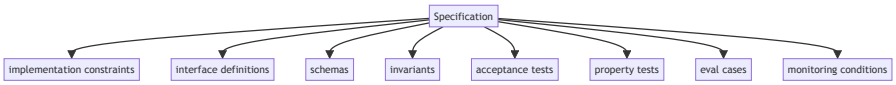
7. Specifications also become machine-checkable

This may be the most important difference.

Traditional requirements are often ultimately evaluated by asking:

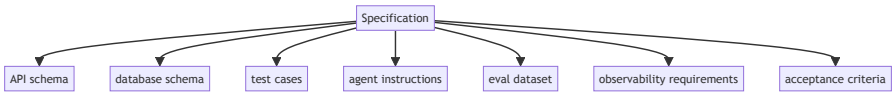
> "Does this satisfy the customer's requirement?"

Specifications can increasingly be transformed into:



So the specification becomes a **source from which engineering artifacts can be generated**.

For example:



That is considerably more powerful than a conventional requirements document.

---

8. Requirements are often human-facing; specifications increasingly become dual human/machine-facing

A traditional requirement might say:

> "Administrators should be able to disable a user."

A specification might define:

```
Operation: DisableUser

Authorization:
  caller.role == ADMIN

Precondition:
  target.status != DELETED

State transition:
  ACTIVE → DISABLED
  SUSPENDED → DISABLED

Forbidden transitions:
  DELETED → DISABLED
```

```
Side effects:
  invalidate active sessions
  revoke API tokens

Audit:
  emit UserDisabled event

Idempotency:
  disabling an already-disabled user succeeds
  and produces no second state transition

Verification:
  unauthorized caller → 403
  disabled user → sessions invalidated
  repeated request → same final state
```

This is almost halfway between **documentation and executable semantics**.

That's exactly the direction AI-native software engineering is moving toward.

---

9. A useful conceptual distinction

I would formulate it this way for your chapter:

> **Requirements express desired outcomes. Specifications define the observable behavior, constraints, interfaces, invariants, and acceptance conditions that make those outcomes unambiguous and verifiable.**

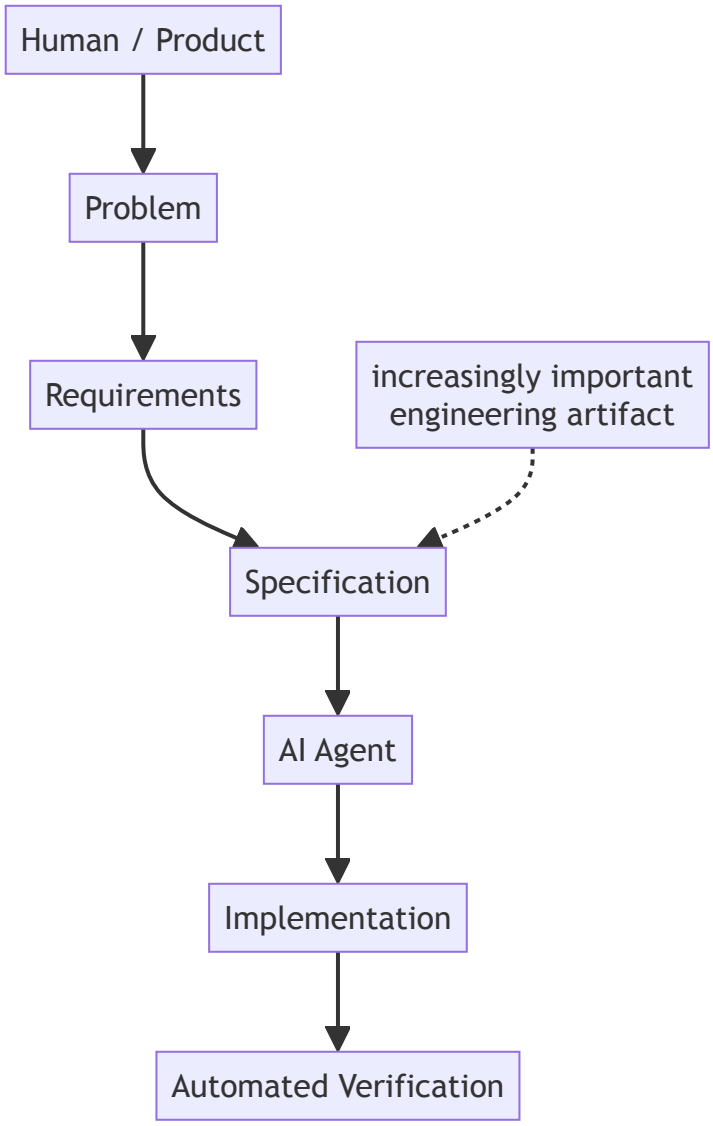
Or even more compactly:

> **Requirements describe intent. Specifications operationalize intent.**

And in the AI-engineering context:

> **The traditional engineer interprets requirements; the AI agent executes against specifications.**

That doesn't mean humans stop defining requirements. Rather, there is a new intermediate engineering activity:



This also explains why **Specification Engineering** may become a distinct skill in the AI-native software development stack.